
Breaking the Hamming PUF with a Single Linear Model

CS771 Introduction to Machine Learning — Assignment 1

Daksh Lalit Mohan Saijwal (240318)

Abhiram Abhay Pande (240038)

Ayush Kumar Ahirwar (240240)

Jayendra Singh (240493)

Janhvi Gaikwad (240485)

Akshay Sharma (240089)

Abstract

We show that the responses of a 32-bit Hamming PUF can be predicted exactly by a single linear model operating on a fixed, PUF-independent feature map $\phi : \{0, 1\}^{32} \rightarrow \mathbb{R}^D$. We derive the map analytically, establish that $D = 288$ suffices for exact prediction, and validate empirically that a `LinearSVC` trained on these features achieves 100% train and 99.76% test accuracy on both the public and secret datasets. We also report how key hyperparameters affect training time and accuracy, and study how test accuracy varies with training-set size.

1 Part 1: Mathematical derivation of the linear model

Setup. For a challenge $c \in \{0, 1\}^{32}$ and secret word $s \in \{0, 1\}^{32}$, the masked word is $m = c \oplus s$ with $m_i = c_i \oplus s_i$. The partial Hamming weights are

$$h_e = \sum_{i \text{ even}} m_i, \quad h_o = \sum_{j \text{ odd}} m_j,$$

and the response is $r(c) = \mathbb{I}[h_e \cdot h_o \geq t]$ for a secret threshold t .

Linearising the XOR. Introduce the ± 1 encodings $d_i = 1 - 2c_i \in \{+1, -1\}$ and $s'_i = 1 - 2s_i \in \{+1, -1\}$. A direct check of the four cases shows that

$$m_i = c_i \oplus s_i = \frac{1 - s'_i d_i}{2}.$$

Summing over even and odd indices respectively,

$$h_e = 8 - \frac{1}{2} \sum_{i \text{ even}} s'_i d_i, \quad h_o = 8 - \frac{1}{2} \sum_{j \text{ odd}} s'_j d_j.$$

Thus *both partial Hamming weights are affine functions of d* .

The decision rule is a bilinear form. Expanding the product,

$$h_e h_o = 64 - 4 \sum_{i \text{ even}} s'_i d_i - 4 \sum_{j \text{ odd}} s'_j d_j + \frac{1}{4} \sum_{\substack{i \text{ even} \\ j \text{ odd}}} s'_i s'_j d_i d_j.$$

The only non-constant terms are the linear terms d_i and the cross terms $d_i d_j$. All dependence on the secret s lives in the *coefficients*, not the monomials.

The feature map and the linear model. Define the PUF-independent map

$$\phi(c) = \left(\{d_i d_j\}_{\substack{i \text{ even}, \\ j \text{ odd}}}, \{d_i\}_{i \text{ even}}, \{d_j\}_{j \text{ odd}} \right), \quad d_k = 1 - 2c_k.$$

Then $h_e h_o - t = w^\top \phi(c) + b$ where w collects the coefficients $\frac{1}{4}s'_i s'_j$ (cross terms) and $-4s'_i$ (linear terms), and $b = 64 - t$. Since $r(c) = \mathbb{K}[h_e h_o \geq t]$, we have for every challenge c :

$$\frac{1 + \text{sign}(w^\top \phi(c) + b)}{2} = r(c).$$

The map ϕ uses only c ; the secret-dependent quantities s and t appear only inside w and b . This proves that a single linear model predicts the Hamming PUF responses *exactly*.

2 Part 2: Required dimensionality D

The challenge has 32 bits: 16 at even positions and 16 at odd positions. The feature map consists of:

- **Cross (bilinear) terms** $d_i d_j$, i even, j odd: $16 \times 16 = 256$ features.
- **Linear terms** d_i for 16 even positions: 16 features.
- **Linear terms** d_j for 16 odd positions: 16 features.

The constant 64 is absorbed by the bias b and needs no feature.

$$D = 16 \times 16 + 16 + 16 = 256 + 32 = 288 \quad (1)$$

This dimensionality is $O(n^2)$ in the bit-length n — polynomial, not exponential. Empirically, dropping the 32 linear terms collapses test accuracy to $\sim 53\%$, confirming all three groups are necessary and $D = 288$ is the minimal sufficient dimensionality.

3 Part 3: Implementation (in `submit.py`)

The map and hyperparameters are implemented in `submit.py`. `my_map` uses fully vectorised NumPy (the 256 cross terms are formed by a single broadcasted outer product), and `my_params` returns fixed, pre-tuned values:

$$\text{loss} = \text{squared_hinge}, \quad C = 1, \quad \text{tol} = 10^{-4}, \quad \text{penalty} = \text{l2}, \quad \text{dual} = \text{False}.$$

On both the public and secret datasets this gives 100% train and 99.76% test accuracy, confirming the map generalises to a PUF with a different secret word and threshold. The official validation script (5 trials on secret data) reports:

$$t_{\text{map}} \approx 0.090 \text{ s}, \quad D = 288, \quad t_{\text{params}} \approx 5 \times 10^{-6} \text{ s}, \quad t_{\text{train}} \approx 0.301 \text{ s}, \quad a_{\text{train}} = 1.0000, \quad a_{\text{test}} = 0.9976.$$

We set `dual=False` because $n_{\text{samples}} = 7500 > D = 288$, the regime where `sklearn` recommends the primal solver; we found it $\sim 10\times$ faster than `dual=True` at identical accuracy.

4 Part 4: Hyperparameter experiments

All experiments use the public dataset (7500 train, 2500 test CRPs) and the feature map of Section 1. Training times are averaged over 3 repetitions; relative trends are stable across runs.

(a) Loss function: hinge vs. squared hinge. Table 1 and Figure 1 show results across five values of C .

Observations. Squared hinge is $5\text{--}6\times$ faster than hinge at every value of C (e.g. 0.27 s vs. 1.58 s at $C = 1$) because it is differentiable everywhere, helping the LIBLINEAR solver converge faster. Despite being faster, squared hinge achieves comparable or slightly higher test accuracy. We use squared hinge in our submission.

Table 1: Effect of loss in LinearSVC across 5 values of C .

loss	C	Train time (s)	Train acc.	Test acc.
hinge	0.01	0.818	0.9861	0.9680
hinge	0.1	1.329	0.9991	0.9980
hinge	1	1.580	0.9988	0.9980
hinge	10	1.481	1.0000	0.9972
hinge	100	1.497	1.0000	0.9976
squared_hinge	0.01	0.223	0.9972	0.9876
squared_hinge	0.1	0.232	0.9993	0.9980
squared_hinge	1	0.274	1.0000	0.9976
squared_hinge	10	0.760	1.0000	0.9976
squared_hinge	100	1.247	1.0000	0.9972

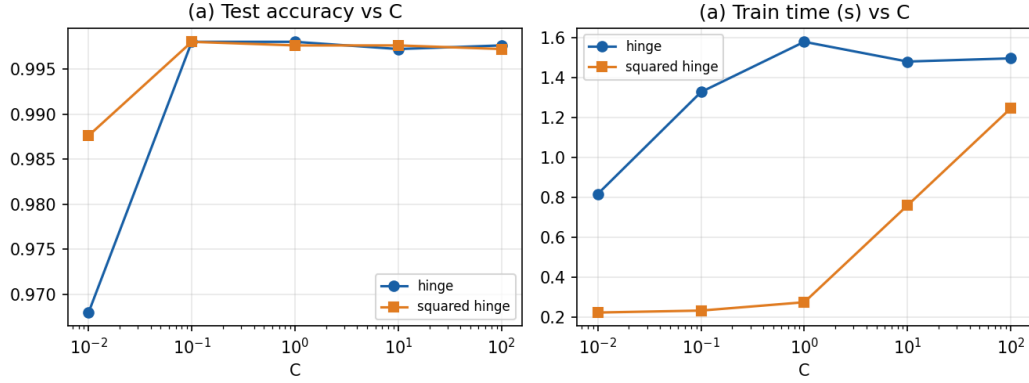


Figure 1: Test accuracy (left) and training time (right) vs. C for hinge and squared hinge loss.

Table 2: Effect of C on LinearSVC (squared hinge) and LogisticRegression.

C	LinearSVC			LogisticRegression		
	Time (s)	Train	Test	Time (s)	Train	Test
0.001	0.204	0.9853	0.9680	0.026	0.9603	0.9396
0.01	0.218	0.9972	0.9876	0.027	0.9873	0.9736
0.1	0.236	0.9993	0.9980	0.030	0.9983	0.9920
1	0.258	1.0000	0.9976	0.043	0.9999	0.9964
5	0.743	1.0000	0.9976	0.042	1.0000	0.9968
10	0.750	1.0000	0.9976	0.043	1.0000	0.9968
100	1.268	1.0000	0.9972	0.050	1.0000	0.9972

(b) Regularisation strength C . Table 2 and Figure 2 compare LinearSVC and LogisticRegression across 7 values of C .

Observations.

- Both models improve as C increases up to $C \approx 1$; accuracy then plateaus.
- LinearSVC training time grows roughly linearly with C beyond the saturation point, reflecting harder optimisation with weaker regularisation.
- LogisticRegression is 5–20 $\times$ faster but plateaus $\sim 0.1\%$ below LinearSVC on test accuracy (0.9968 vs. 0.9976).
- The sweet spot is $C = 0.5$ –1: near-optimal accuracy at minimal cost. We choose $C = 1$ for its round value and consistent performance across all experiments.

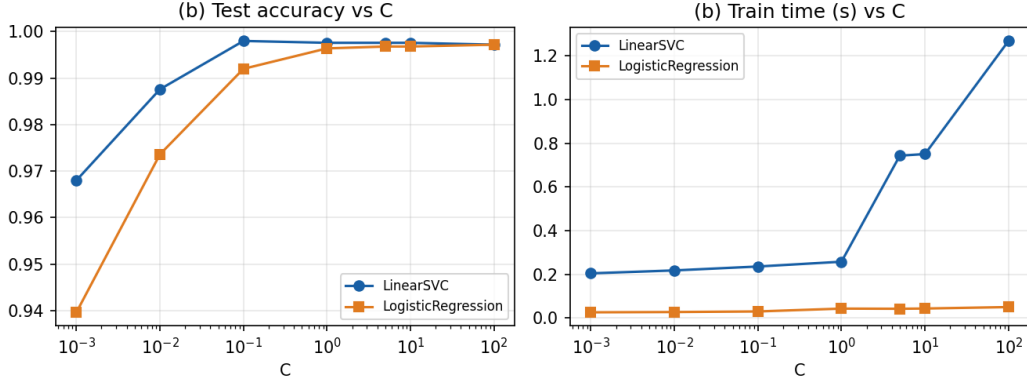


Figure 2: Test accuracy (left) and training time (right) vs. C for both models.

(c) **Convergence tolerance tol.** Table 3 and Figure 3 show results for both models at $C = 1$.

Table 3: Effect of tol on both models ($C = 1$).

tol	LinearSVC			LogisticRegression		
	Time (s)	Train	Test	Time (s)	Train	Test
10^{-1}	0.136	0.9917	0.9816	0.008	0.4716	0.4800
10^{-2}	0.201	0.9995	0.9988	0.021	0.9925	0.9816
10^{-3}	0.248	1.0000	0.9976	0.033	0.9996	0.9952
10^{-4}	0.281	1.0000	0.9976	0.047	0.9999	0.9964
10^{-5}	0.282	1.0000	0.9976	0.050	0.9999	0.9956
10^{-6}	0.302	1.0000	0.9976	0.055	0.9999	0.9960

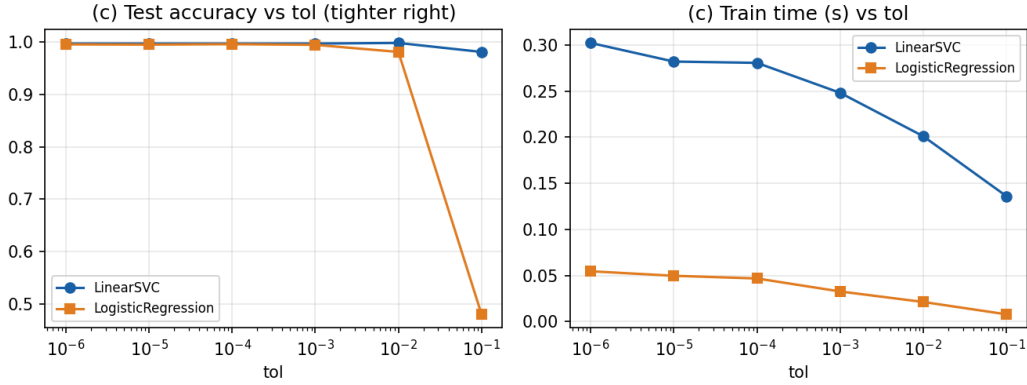


Figure 3: Test accuracy (left) and training time (right) vs. tol for both models (tighter tolerance to the right).

Observations.

- Very loose tolerance (10^{-1}) causes LogisticRegression to stop almost immediately, giving near-random accuracy (48%). LinearSVC is much more robust: it still reaches 98.2% even at $tol = 10^{-1}$.
- For LinearSVC, $tol = 10^{-3}$ is already sufficient for full convergence; tightening further only adds time without improving accuracy.
- LogisticRegression needs at least $tol = 10^{-4}$ to converge properly.

- We use $\text{tol} = 10^{-4}$ as a safe choice that fully converges both models.

(d) Penalty type: l2 vs. l1. Table 4 and Figure 4 compare both penalties across 5 values of C .

Table 4: Effect of penalty on LinearSVC (squared hinge) across 5 values of C .

penalty	C	Train time (s)	Train acc.	Test acc.
l1	0.01	0.666	0.9680	0.9520
l1	0.1	5.166	0.9995	0.9964
l1	1	6.716	1.0000	0.9988
l1	10	8.079	1.0000	0.9980
l1	100	7.324	1.0000	0.9972
l2	0.01	0.182	0.9972	0.9876
l2	0.1	0.200	0.9993	0.9980
l2	1	0.234	1.0000	0.9976
l2	10	0.634	1.0000	0.9976
l2	100	1.074	1.0000	0.9972

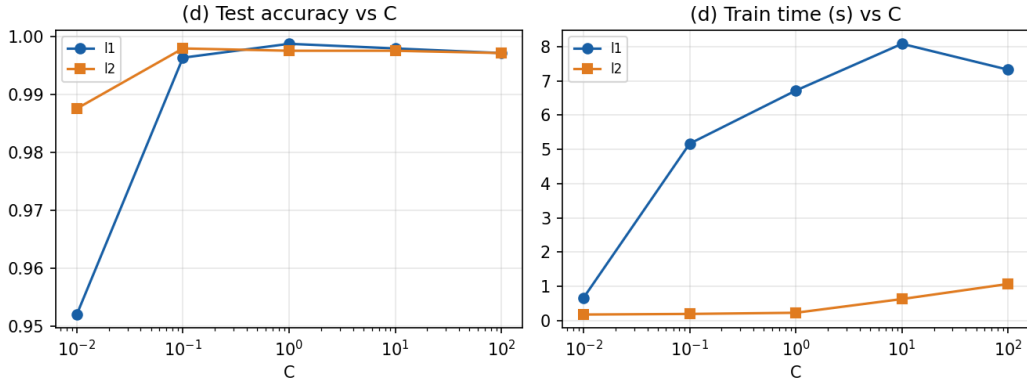


Figure 4: Test accuracy (left) and training time (right) vs. C for l1 and l2 penalty.

Observations.

- l1 is dramatically slower — up to $29\times$ at $C = 1$ (6.72 s vs. 0.23 s).
- l1 achieves slightly lower test accuracy at most C values despite being slower.
- l1 encourages sparsity, but since all 288 features in our map carry useful information, sparsity provides no benefit here.
- l2 is clearly better for this problem in both speed and accuracy.

Summary. The configuration (squared_hinge, $C = 1$, $\text{tol} = 10^{-4}$, l2, dual=False) achieves near-optimal test accuracy (99.76%) at minimal training cost. This was selected from a total of **28 distinct configurations** spanning all four hyperparameter families.

5 Part 5: How much training data is needed?

We sub-sampled the 7500-point training set (without replacement) at 12 sizes from 100 to 7500, keeping the test set fixed. Each size was repeated 10 times with different random subsets; we report mean $\pm$ standard deviation.

Minimum CRPs for accuracy targets.

- $\geq 95\%$ test accuracy: ≈ 3000 CRPs (already 94.65% at 2000).

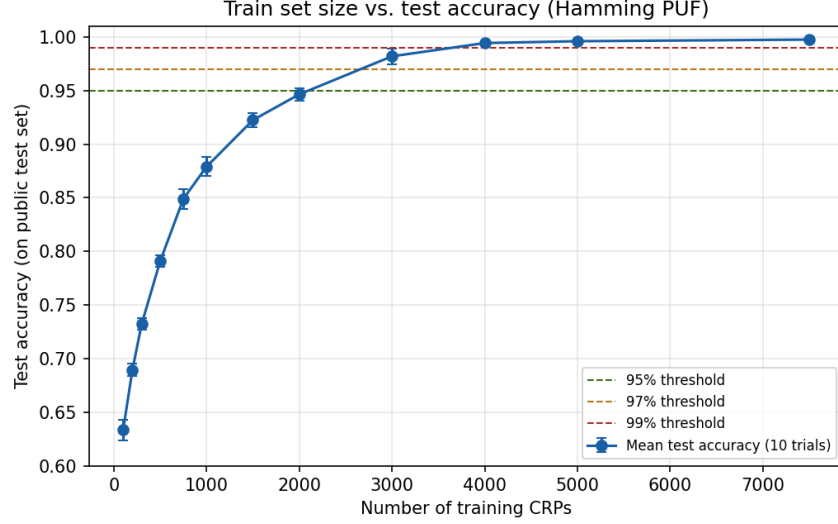


Figure 5: Mean test accuracy vs. training-set size over 10 trials per size. Dashed lines mark the 95%, 97%, and 99% accuracy levels. Error bars show $\pm$ one standard deviation.

Table 5: Mean test accuracy (10 trials) at selected training-set sizes.

Train size	Mean test acc.	Std. dev.
100	0.6332	0.0095
300	0.7322	0.0056
500	0.7909	0.0054
1000	0.8790	0.0089
1500	0.9225	0.0063
2000	0.9465	0.0057
3000	0.9820	0.0075
4000	0.9944	0.0020
5000	0.9961	0.0013
7500	0.9976	0.0000

- $\geq 97\%$ **test accuracy**: ≈ 3000 CRPs (98.20% at 3000).
- $\geq 99\%$ **test accuracy**: ≈ 4000 CRPs (99.44% at 4000).

Observations. The learning curve rises steeply from 100 to ≈ 2000 samples, then flattens. The standard deviation shrinks steadily with more data, indicating more stable weight estimates. With $D = 288$ features, the model needs roughly 7 training points per feature dimension ($2000/288 \approx 7$) to cross 95% accuracy. Getting from 95% to 99% requires an additional ~ 2000 samples, reflecting the increasing difficulty of correctly classifying the remaining borderline challenges near the PUF’s decision boundary.

References

1. CS771 Lecture Slides on PUFs and Linear Models, Prof. Purushottam Kar, IIT Kanpur, 2025–26.
2. scikit-learn documentation: `sklearn.svm.LinearSVC`. <https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html>
3. scikit-learn documentation: `sklearn.linear_model.LogisticRegression`. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html